Sumario

1 - MongoDB como gestor de base de datos NoSQL....2
Ventajas....2
Desventajas....2
2 - Instalación de MongoDB....3
3 - Elementos esenciales de MongoDB: Base De Datos - Colección - Documento....6
Acotaciones....10
4 - Insertar documentos mediante los métodos insertOne e insertMany de una colección....10
5 - Campo obligatorio _id....12
6 - Comandos del shell de MongoDB: use (nombre base datos), show dbs, show collections, help.15
7 - Borrar bases de datos, colecciones o todos los documentos de una colección....17
8 - Recuperar algunos documentos de una colección con el método find....19
9 - Operadores relacionales $eq, $gt, $gte, $lt, $lte, $in, $nin y $ne....22
Listado de operadores relacionales....24
10 - Borrar documentos de una colección con los métodos deleteOne y deleteMany....25
11 - Modificar un documento mediante el método updateOne....27
12 - Modificar múltiples documentos con el método updateMany....31
13 - Operadores lógicos $and, $or y $not....34
15 - Recuperar algunos campos de una colección con el método find....36
16 - Documentos embebidos: definición de campos de tipo documento....38
17 - Documentos embebidos: definición de campos de arreglo con elementos de tipo documento. .41
Consultas de los documentos embebidos....44
Acotaciones....46
18 - Campo _id generado por MongoDB....46
19 - Tipo de dato Date en MongoDB....48
Problema....49
20 - Tipo de dato Binary Data....51

1 - MongoDB como gestor de base de datos NoSQL

El gestor de base de datos MongoDB se lo puede asociar a un conjunto de gestores de bases de datos que no tienen como lenguaje principal el SQL para su manipulación.

Los gestores de bases de datos NoSQL no requieren estructuras fijas como tablas, normalmente no soportan operaciones join y presentan como gran ventaja que pueden escalar en forma sencilla.

Ventajas

- Permiten escalar en forma sencilla.
- Tienen un lenguaje adaptada al modelo de datos que implementa el gestor de base de datos.
- Permiten administrar grandes cantidades de datos no estructurados.

Desventajas

- No hay al momento gran cantidad de desarrolladores que conozcan este tipo de gestores de bases de datos, a diferencia a lo que ocurre con los gestores clásicos como Oracle, SQL Server, MySQL, etc.
- La compatibilidad entre los distintos gestores de bases de datos NoSQL es nula.
- Los gestores NoSQL son una tecnología relativamente nueva por lo que le falta alguna madurez a algunos de ellos.
- La cantidad de herramientas para administrarlos por el momento es muy limitado.

Los gestores NoSQL más destacados:

- MongoDB
- Cassandra
- redis
- CouchDB
- Amazon SimpleDB
- IBM Informix
- Elasticsearch
- Hadoop
- Cloud Datastore (Google)

Estos y muchos otros gestores de bases de datos son alternativas a los gestores de bases de datos relacionales tradicionales (SQL Server, MySQL, Oracle etc.) y pretenden superarlos en velocidad de acceso a los datos, manejo de grandes volúmenes de datos y la posibilidad de tener sistemas distribuidos.

Por el momento no hay un claro ganador en este movimiento de los gestores de bases de datos NoSQL, pero podemos ver que grandes empresas como Google, Amazon, Ibm etc. están invirtiendo en este sector de la tecnología.

El sitio oficial de MongoDB define a su producto: "MongoDB es una base de datos de documentos que ofrece una gran escalabilidad y flexibilidad, y un modelo de consultas e indexación avanzado."

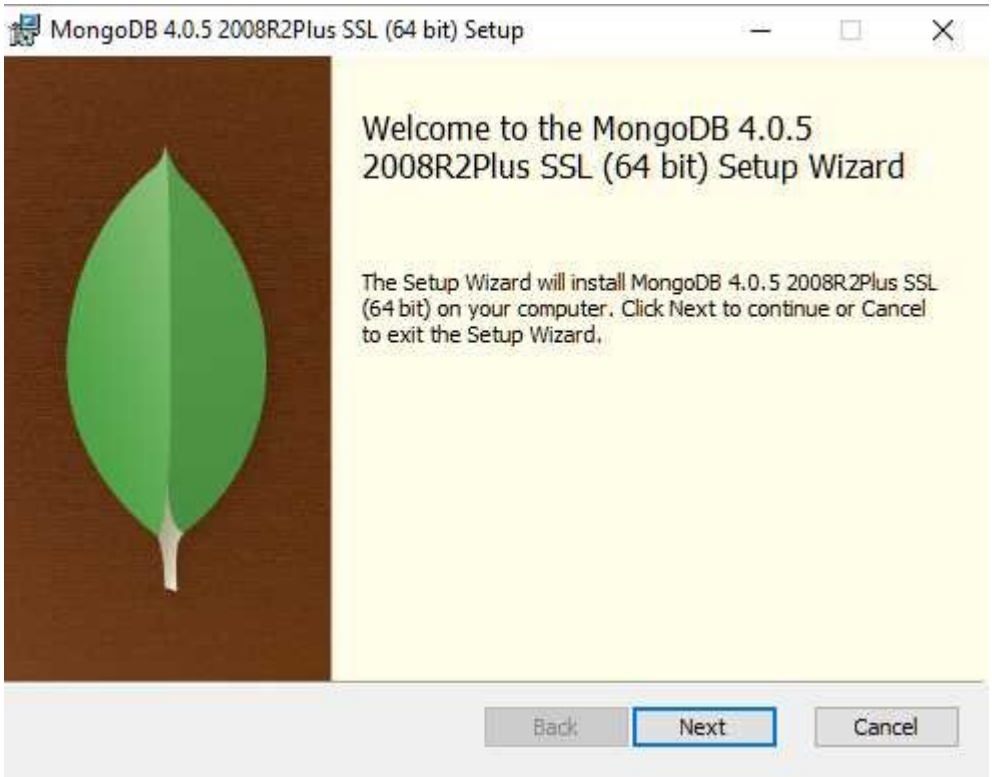
Sus características son:

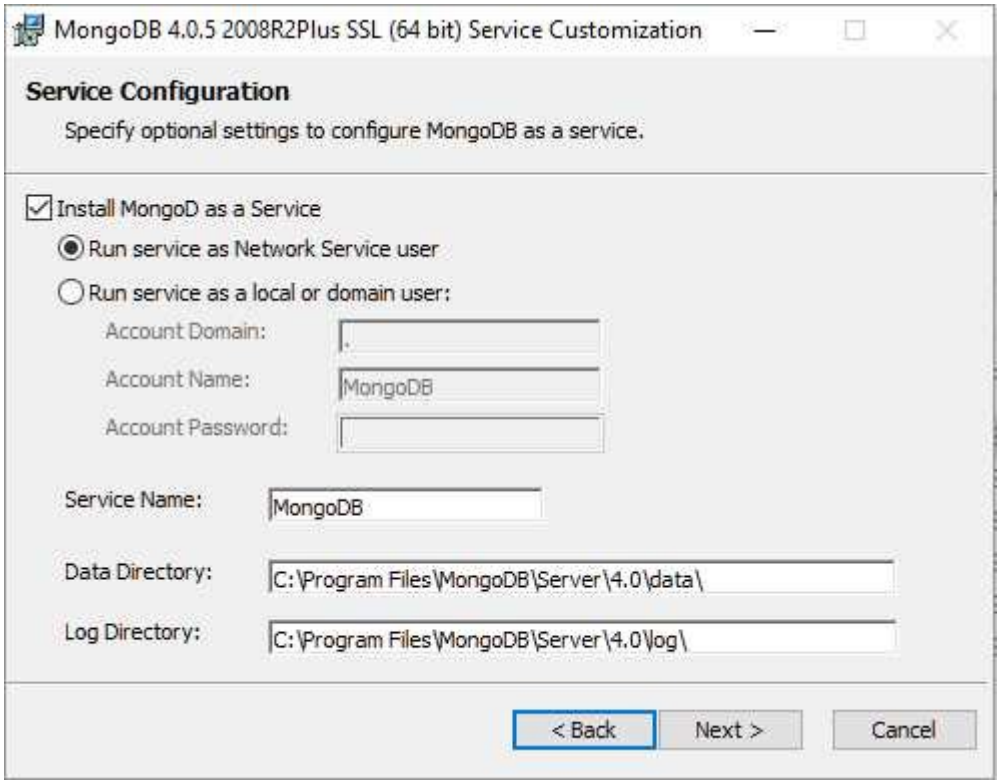
- MongoDB almacena datos en documentos JSON,es decir, cada documento puede contener diferentes campos y las estructuras de datos se pueden ir modificando.
- MongoDB es una base de datos distribuida,por lo que es fácil de usar y proporciona una elevada disponibilidad, escalabilidad horizontal y distribución geográfica.
- MongoDB es una base de datos de código abierto y de uso gratuito.

2 - Instalación de MongoDB

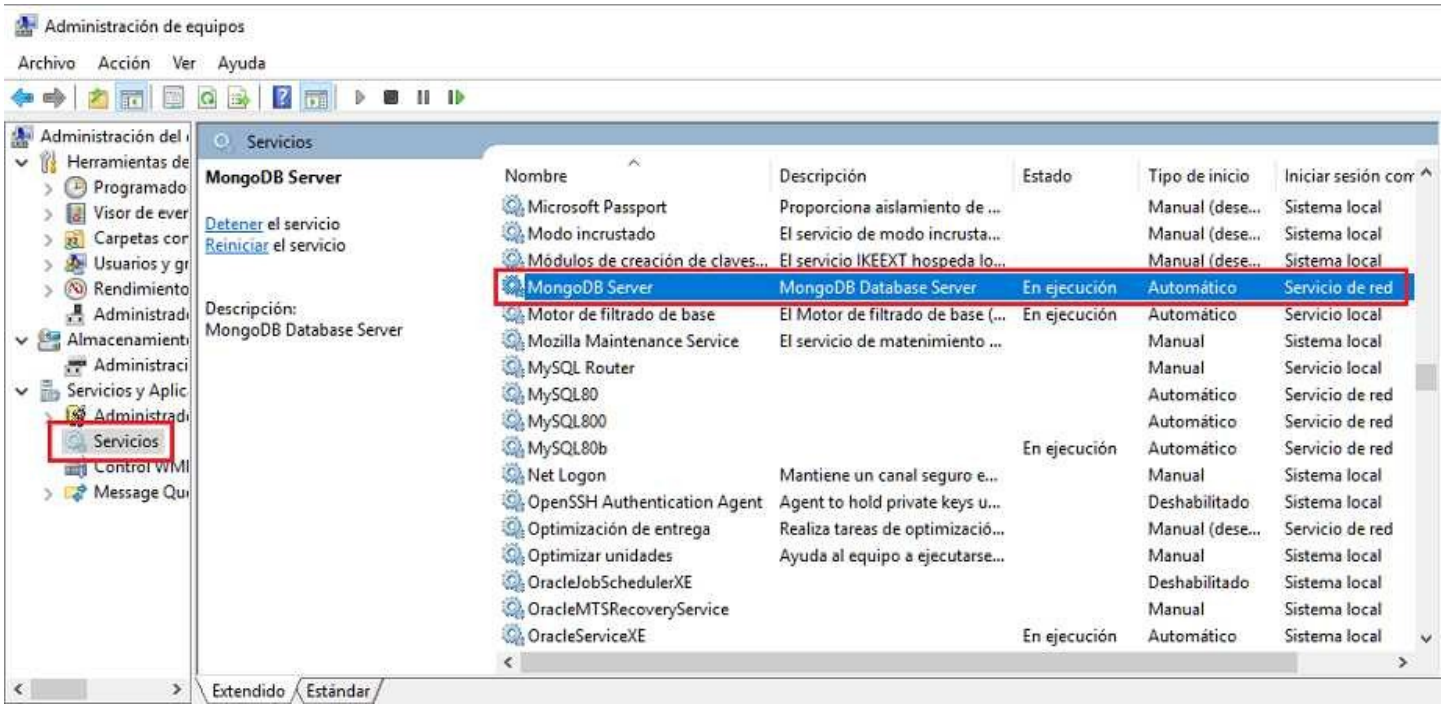
El primer paso será descargar el motor de base de datos MongoDB de su sitio oficial: MongoDB.

Procedemos seguidamente a ejecutar su instalador:

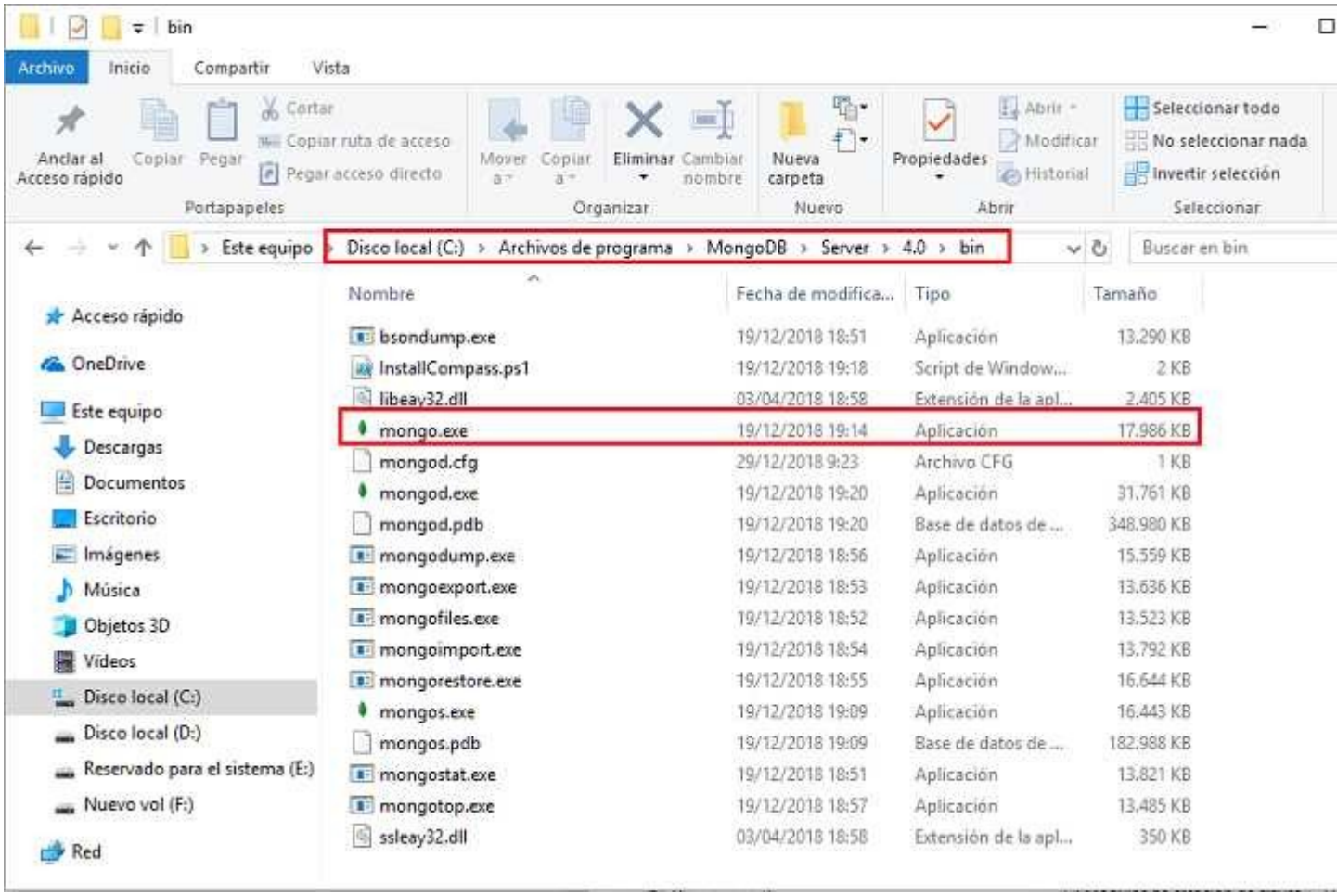




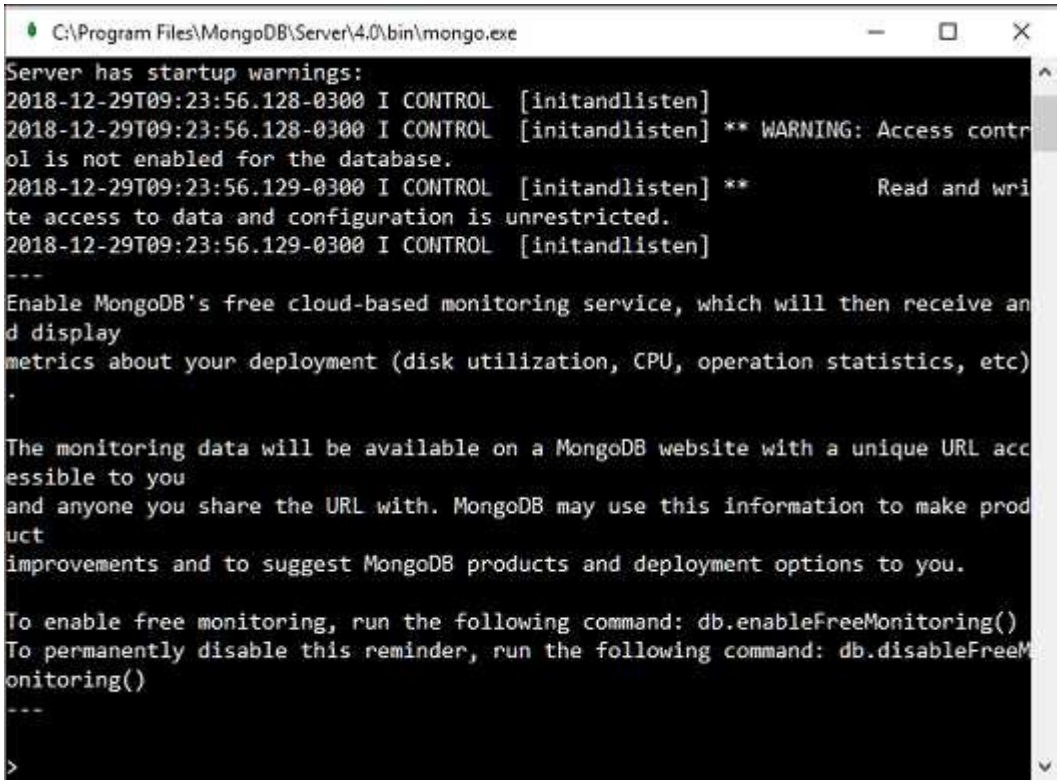
Una vez instalado MongoDB el servidor de base de datos se inicia cada vez que encendemos la computadora, podemos comprobar ingresando al "Administrador de equipos" a la sección de "Servicios":



Utilizaremos en los primeros conceptos una aplicación de consola que se instala con MongoDB que nos permite interactuar con el gestor de base de datos y sus elementos. Debemos ejecutar el programa mongo.exe:

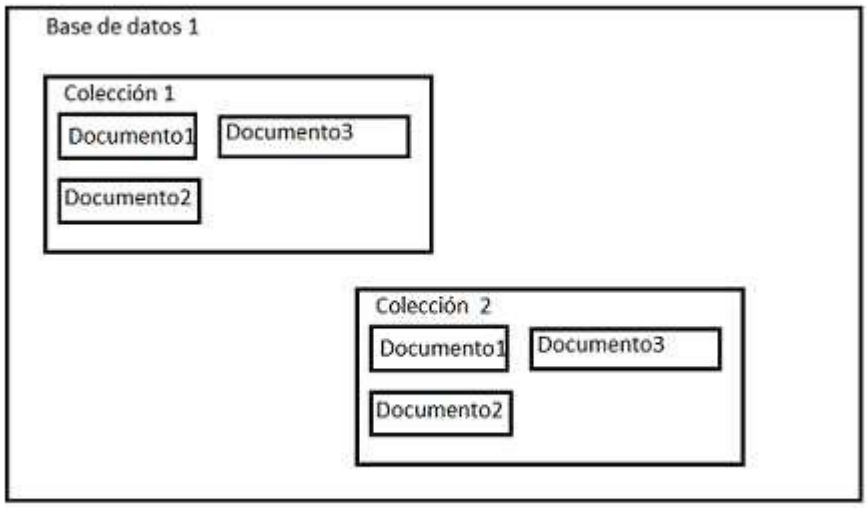


Tenemos una interfaz en modo texto donde podemos escribir los distintos comandos de MongoDB:



3 - Elementos esenciales de MongoDB: Base De Datos - Colección - Documento

Software de aplicación
Mongo
pdf
lenguajes
MongoDB
PDF
aplicación
Node.js
gestor de base de datos
base de datos
MongoDB es una base de datos documental, el elemento esencial es el documento que normalmente se los agrupa en colecciones de documentos similares. Una base de datos en MongoDB es un conjunto de colecciones:



MongoDB utiliza el formato JSON para representar los datos en un "documento":

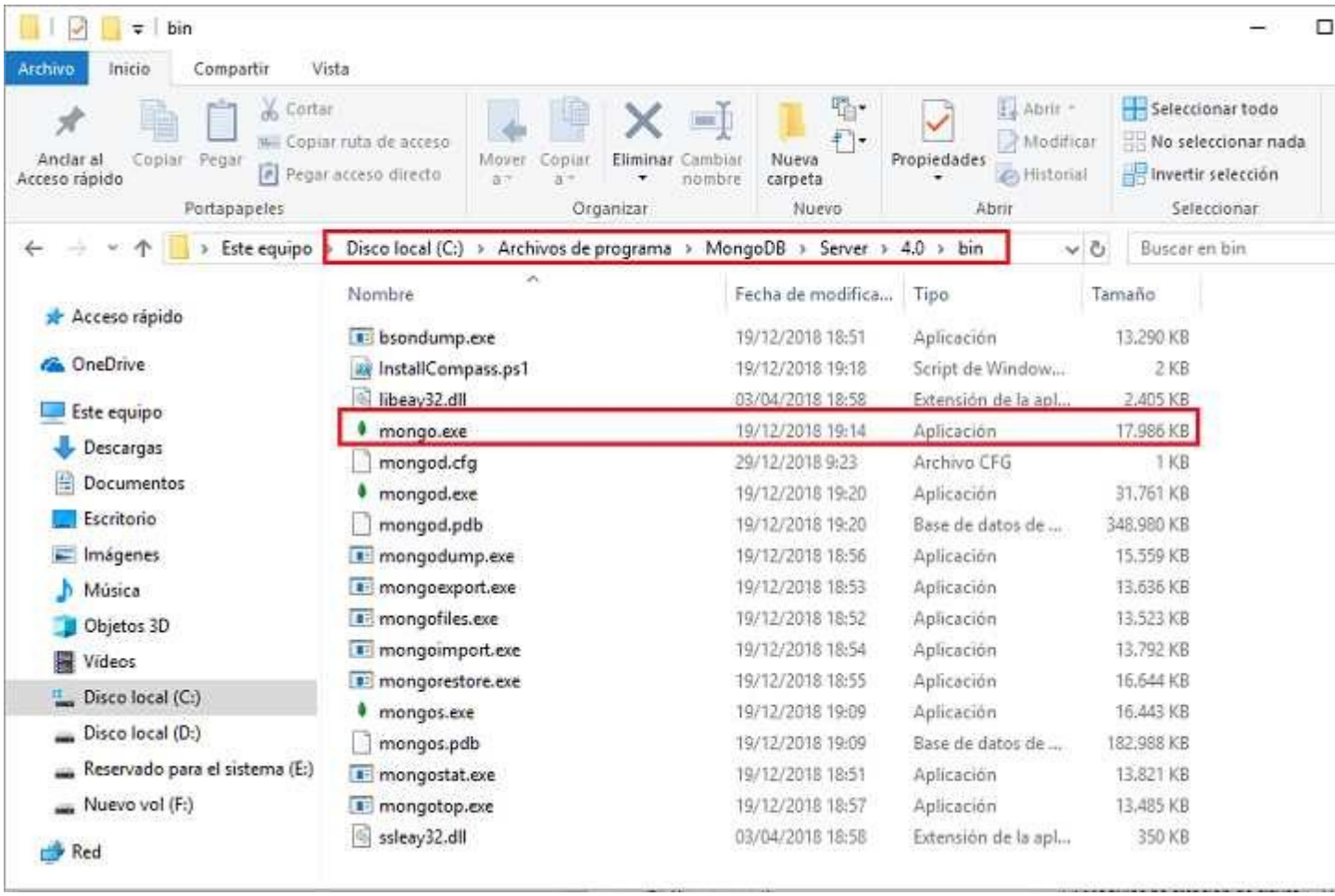
```
{
  codigo: 1,
  nombre: 'El aleph',
  autor: 'Borges',
  editoriales: ['Planeta','Siglo XXI']
}
```

El documento anterior representa un libro donde se almacenan su código, nombre, autor y editoriales que lo suministran.

Si uno ha trabajado mucho con gestores de bases de datos relacionales puede relacionar este concepto de "documento" con el de registro de una tabla, veremos más adelante que un documento en MongoDB es mucho más flexible y desestructurado.

Un consejo es tratar de no comparar constantemente los gestores de bases de datos relacionales con MongoDB, mirar sin prejuicios las bondades que aportan los gestores de bases de datos documentales e imaginar las ventajas que nos aportan cuando debemos almacenar y recuperar datos.

Ingresemos al programa que viene por defecto en Mongo para interactuar con el gestor de base de datos:



Crear primero nuestra primer base de datos:

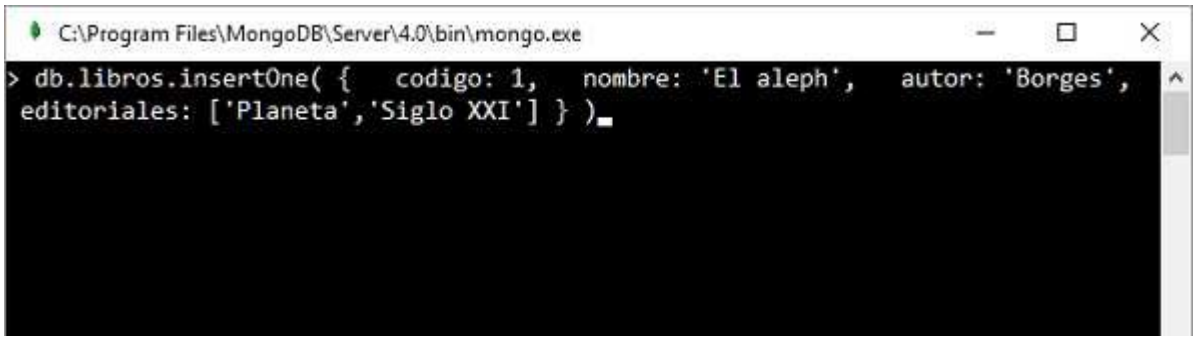
Mediante el comando use activamos una base de datos existente o creamos una nueva (en nuestro caso la llamamos base1), queda luego activa la base de datos "base1".

Procedemos ahora a crear la colección libros e insertar el primer documento, la colección se crea en el momento que insertamos el primer documento:

```
db.libros.insertOne(
  {
    codigo: 1,
    nombre: 'El aleph',
```

```
    autor: 'Borges',
    editoriales: ['Planeta','Siglo XXI']
  }
)
```

En el shell de Mongo tenemos que ingresar:



Luego de esto ya se ha creado la colección "libros" y se ha insertado el primer documento.

Procedamos a insertar el segundo documento:

```
db.libros.insertOne(
  {
    codigo: 2,
    nombre: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editoriales: ['Planeta']
  }
)
```

Tengamos en cuenta que mediante el objeto "db" accedemos a la base de datos activa (la misma la activamos con el comando use base1), seguidamente disponemos el nombre de la colección "libros" y finalmente el nombre del método "insertOne" al que le pasamos un objeto en formato JSON.

Ahora nuestra colección "libros" tiene dos documentos, si queremos mostrar los datos almacenados en la colección "libros" podemos llamar al método "find" sin pasar parámetros al mismo:

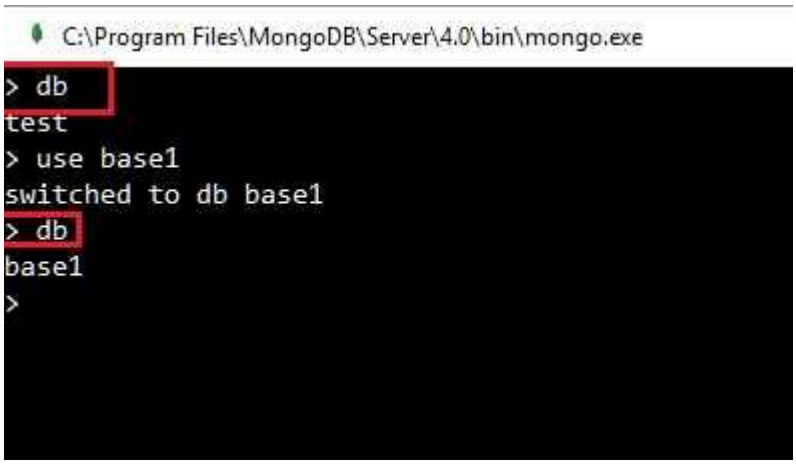
Los datos que vemos coinciden con los ingresados al llamar al método insertOne, con la salvedad que se ha agregado un campo llamado _id en forma automática.

Todos los documentos requiere una clave principal almacenada en el campo _id. Podemos indicar nosotros el valor a almacenar en el campo _id, pero si no lo hacemos se crea en forma automática.

aplicación
Mongo
Python
motor de base de datos
Base de datos
lenguajes
PDF
Phyton
pdf

Acotaciones

Cada vez que iniciamos MongoDB shell se activa por defecto la base de datos 'test' mediante el comando 'use' debemos activar la base de datos que necesitamos trabajar. Para saber en todo momento que base de datos se encuentra activa debemos escribir la variable 'db':



4 - Insertar documentos mediante los métodos insertOne e insertMany de una colección

Vimos en el concepto anterior como crear una base de datos documental en MongoDB, crear una colección e insertar un documento mediante el método insertOne.

Para inserta un documento o un conjunto de documentos disponemos de los métodos:

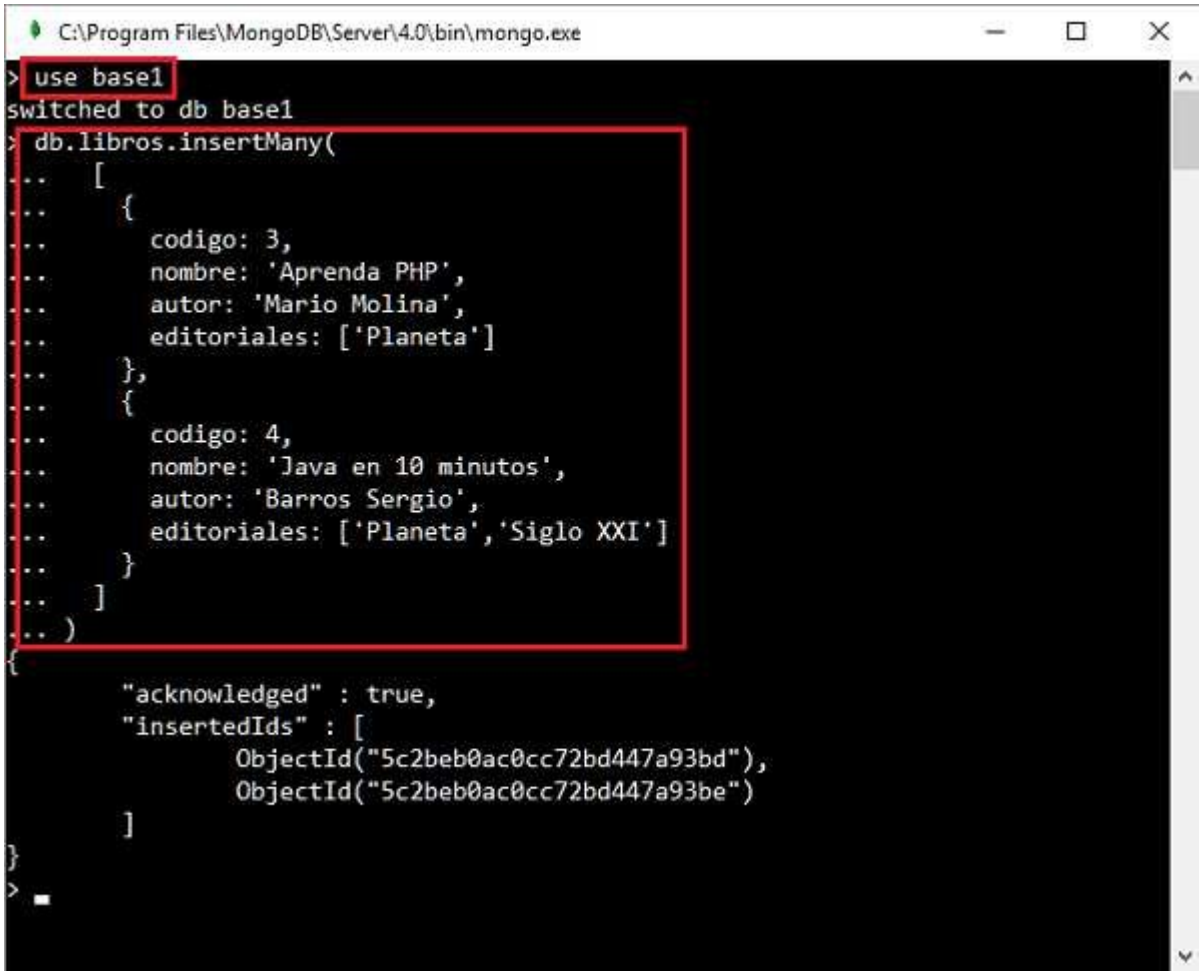
- insertOne: Inserta un documento en una colección.
- insertMany: Inserta múltiples documentos en una colección.

Procedamos a insertar más de un documento en la colección "libros" mediante el método insertMany:

```
db.libros.insertMany(
  [
    {
```

```
      codigo: 3,
      nombre: 'Aprenda PHP',
      autor: 'Mario Molina',
      editoriales: ['Planeta']
    },
    {
      codigo: 4,
      nombre: 'Java en 10 minutos',
      autor: 'Barros Sergio',
      editoriales: ['Planeta','Siglo XXI']
    }
  ]
)
```

En la consola de MongoDB tenemos como resultado:



Tengamos en cuenta que si recién activamos la consola debemos activar la base de datos "base1" mediante el comando "use":

```
use base1
```

Luego llamamos al método "insertMany" y le pasamos un array con todos los documentos a almacenar en la colección "libros".

Tener en cuenta que utilizamos la consola de MongoDB (shell) con el objetivo a aprender los comandos esenciales, luego en la realidad estos datos serán enviados desde nuestras aplicaciones que podrán estar escritas en Python, Ruby, C#, Java etc.

Podemos borrar el contenido de la consola de MongoDB (shell) mediante el comando:

```
cls
```

El mismo resultado lo podemos obtener presionando las teclas: CTRL + L.

Mostremos los documentos almacenados en la colección "libros" mediante el método "find":

Como podemos observar al ejecutar el método "find" nuestra colección "libros" tiene almacenado 4 documentos.

5 - Campo obligatorio _id

En MongoDB todo documento requiere un campo clave que se debe llamar _id. Si nosotros como desarrolladores no definimos dicho campo el mismo se crea en forma automática y se carga un valor único.

Podemos definir y cargar un valor en el campo _id cuando inicializamos un documento:

```
db.clientes.insertOne(
  {
    _id: 1,
    nombre: 'Lopez Marcos',
    domicilio: 'Colon 111',
    provincia: 'Cordoba'
  }
)
```

Cuando ejecutamos la inserción desde el shell de MongoDB tenemos como resultado:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> use base1
switched to db base1
> db.clientes.insertOne(
...   {
...     _id: 1,
...     nombre: 'Lopez Marcos',
...     domicilio: 'Colon 111',
...     provincia: 'Cordoba'
...   }
... )
{ "acknowledged" : true, "insertedId" : 1 }
>
```

Cuando se ejecuta el método insertOne nos retorna un objeto JSON informando del resultado de la inserción mediante un objeto con dos campos, el primero acknowledged que indica si el documento fue admitido en la colección y el _id que en este caso lo define el usuario de la base de datos.

Si se produce un error nos genera un objeto JSON con otra estructura, probemos de intentar de ingresar un segundo documento con el mismo _id:

```
db.clientes.insertOne(
  {
    _id: 1,
    nombre: 'Perez Ana',
    domicilio: 'San Martin 222',
    provincia: 'Santa Fe'
  }
)
```

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.clientes.insertOne(
...     {
...       _id: 1,
...       nombre: 'Perez Ana',
...       domicilio: 'San Martin 222',
...       provincia: 'Santa Fe'
...     }
... )
2019-01-02T09:14:49.708-0300 E QUERY    [js] WriteError: E11000 du
plicate key error collection: base1.clientes index: _id_ dup key:
{ : 1.0 } :
WriteError({
        "index" : 0,
        "code" : 11000,
        "errmsg" : "E11000 duplicate key error collection: base1.c
lientes index: _id_ dup key: { : 1.0 }",
        "op" : {
                "_id" : 1,
                "nombre" : "Perez Ana",
                "domicilio" : "San Martin 222",
                "provincia" : "Santa Fe"
        }
})
WriteError@src/mongo/shell/bulk_api.js:461:48
Bulk/mergeBatchResults@src/mongo/shell/bulk_api.js:841:49
Bulk/executeBatch@src/mongo/shell/bulk_api.js:906:13
Bulk/this.execute@src/mongo/shell/bulk_api.js:1150:21
DBCollection.prototype.insertOne@src/mongo/shell/crud_api.js:252:9

@(shell):1:1
>
```

Nos retorna un objeto JSON que entre otros campos define uno llamado errmsg con el mensaje de error.

Si nuestra aplicación administra el campo '_id' hay que tener en cuenta que nunca puede repetirse y en el caso que intentemos ingresar un documento con clave repetida luego dicho documento no se inserta en la colección.

Mostremos todos los documentos almacenados en la colección libros:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.clientes.find()
{ "_id" : 1, "nombre" : "Lopez Marcos", "domicilio" : "Colon 111",
"provincia" : "Cordoba" }
>
```

Hay uno solo ya que el segundo intento no se cargó.

6 - Comandos del shell de MongoDB: use (nombre base datos), show dbs, show collections, help

Ya vimos en conceptos anteriores que mediante el comando use activamos la base de datos que queremos trabajar:

```
use base1
```

Para conocer todas las bases de datos del servidor de MongoDB utilizamos el comando:

```
show dbs
```

Tenemos un resultado similar a:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> show dbs
admin   0.000GB
base1   0.000GB
config  0.000GB
local   0.000GB
test    0.000GB
>
```

Si creamos en conceptos anteriores la base de datos "base1", la misma debe aparecer al ejecutar "show dbs".

Activemos la base de datos "base1" mediante el comando "use":

```
use base1
```

Y seguidamente llamemos al comando "show collections":

```
show collections
```

Tenemos un resultado similar a:

La base de datos "base1" contiene dos colecciones llamadas "clientes" y "libros" que nosotros creamos en conceptos anteriores.

Si queremos consultar los comandos esenciales del shell de MongoDB debemos emplear el comando "help":

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> help
        db.help()                    help on db methods
        db.mycoll.help()             help on collection methods
        sh.help()                    sharding helpers
        rs.help()                    replica set helpers
        help admin                   administrative help
        help connect                 connecting to a db help
        help keys                    key shortcuts
        help misc                    misc things to know
        help mr                      mapreduce

        show dbs                     show database names
        show collections             show collections in current database
        show users                   show users in current database
        show profile                 show most recent system.profile entries with time >=
        show logs                    show the accessible logger names
        show log [name]              prints out the last segment of log in memory, 'global
        use <db_name>                set current database
        db.foo.find()                list objects in collection foo
        db.foo.find( { a : 1 } )     list objects in foo where a == 1
        it                           result of the last line evaluated; use to further ite
        DBQuery.shellBatchSize = x   set default number of items to display on shell
        exit                         quit the mongo shell
>
```

7 - Borrar bases de datos, colecciones o todos los documentos de una colección

Hemos visto como se crea una base de datos, una colección y se insertan documentos en la misma.

Si queremos eliminar todos los documentos de una colección debemos utilizar el método "deleteMany" aplicado a una colección existente:

```
use base1
db.libros.deleteMany({})
show collections
```

Debemos pasar un objeto vacío que se indica con las llaves abiertas y cerradas {}. Luego veremos que podemos borrar solo los documentos que cumplen cierta condición.

Es importante notar que luego de llamar al método deleteMany la colección "libros" sigue existiendo:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> use base1
switched to db base1
> db.libros.deleteMany({})
{ "acknowledged" : true, "deletedCount" : 2 }
> show collections
clientes
libros
>
```

Para eliminar los documentos de una colección y la colección propiamente dicha debemos emplear el método "drop":

```
use base1
db.libros.drop()
show collections
```

Luego de llamar al método drop de la colección "libros" la misma deja de existir:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> use base1
switched to db base1
> show collections
clientes
libros
usuarios
> db.libros.drop()
true
> show collections
clientes
usuarios
>
```

Para eliminar una base de datos en forma completa, es decir todas sus colecciones y documentos debemos emplear el método dropDatabase del objeto "db":

```
show dbs
use base1
db.dropDatabase()
show dbs
```

El método dropDatabase elimina la base de datos activa:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> show dbs
admin   0.000GB
base1   0.000GB
config  0.000GB
local   0.000GB
test    0.000GB
> use base1
switched to db base1
> db.dropDatabase()
{ "dropped" : "base1", "ok" : 1 }
> show dbs
admin   0.000GB
config  0.000GB
local   0.000GB
test    0.000GB
>
```

8 - Recuperar algunos documentos de una colección con el método find

Vimos en conceptos anteriores que podemos recuperar todos los documentos de una colección mediante el método find:

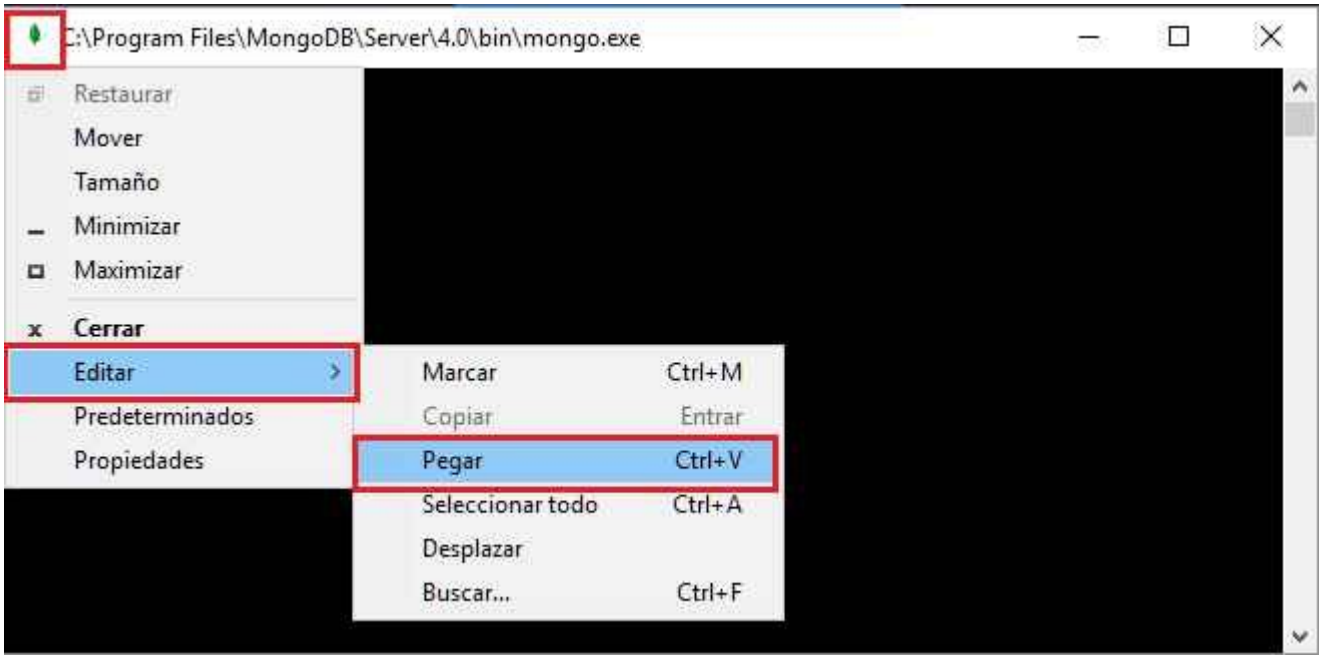
```
use base1
db.libros.drop()

db.libros.insertOne(
  {
    _id: 1,
    titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
    cantidad: 50
  }
)
db.libros.insertOne(
  {
    _id: 2,
    titulo: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
    precio: 50,
    cantidad: 12
  }
)
db.libros.insertOne(
  {
```

```
    _id: 3,
    titulo: 'Aprenda PHP',
    autor: 'Mario Molina',
    editorial: ['Siglo XXI','Planeta'],
    precio: 50,
    cantidad: 20
  }
)
db.libros.insertOne(
  {
    _id: 4,
    titulo: 'Java en 10 minutos',
    editorial: ['Siglo XXI'],
    precio: 45,
    cantidad: 1
  }
)

db.libros.find()
```

Puede evitar escribir todo el código anterior copiando todo el código anterior y desde el menú del shell de MongoDB pegar el bloque con la opción:



El resultado de ejecutar el bloque de comandos es:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
...     _id: 4,
...     titulo: 'Java en 10 minutos',
...     editorial: ['Siglo XXI'],
...     precio: 45,
...     cantidad: 1
...   }
... )
{ "acknowledged" : true, "insertedId" : 4 }
>
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" : [ "Siglo XXI
", "Planeta" ], "precio" : 20, "cantidad" : 50 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "editorial" :
[ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "editorial" : [ "
Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo XXI" ], "prec
io" : 45, "cantidad" : 1 }
>
```

El método find nos permite seleccionar solo algunos documentos que cumplen una condición:

```
db.libros.find({_id : 1})
```

Rescatamos el documento que almacena en el campo '_id' el valor 1:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.libros.find({ _id: 1 })
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "e
ditorial" : [ "Siglo XXI", "Planeta" ], "precio" : 20, "ca
ntidad" : 50 }
>
```

Si pasamos un valor para el campo '_id' que no existe luego el método find no regresa un documento.

Podemos rescatar todos los libros que tiene un precio igual a 50:

```
db.libros.find({precio : 50 })
```

Luego se recuperan dos documentos que cumplen la condición:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.libros.find({precio : 50 })
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "editorial" :
 [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "editorial" : [ "
Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
>
```

Podemos disponer más de un campo:

```
db.libros.find({precio : 50, cantidad : 20 })
```

Solo hay un documento que cumple estas dos condiciones:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.libros.find({precio : 50, cantidad : 20 })
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "editorial" : [ "
Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
>
```

9 - Operadores relacionales $eq, $gt, $gte, $lt, $lte, $in, $nin y $ne

En el concepto anterior vimos como podemos seleccionar mediante el método find algunos documentos que cumplen una condición.

Recordemos con un ejemplo como recuperar todos los libros con un precio igual a 50:

```
use base1
db.libros.drop()

db.libros.insertOne(
  {
    _id: 1,
    titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
    cantidad: 50
  }
)
db.libros.insertOne(
  {
```

```
    _id: 2,
    titulo: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
    precio: 50,
    cantidad: 12
  }
)
db.libros.insertOne(
  {
    _id: 3,
    titulo: 'Aprenda PHP',
    autor: 'Mario Molina',
    editorial: ['Siglo XXI','Planeta'],
    precio: 50,
    cantidad: 20
  }
)
db.libros.insertOne(
  {
    _id: 4,
    titulo: 'Java en 10 minutos',
    editorial: ['Siglo XXI'],
    precio: 45,
    cantidad: 1
  }
)
```

```
db.libros.find({ precio: 50 })
```

Es decir que cuando llamamos al método find pasamos un objeto literal pasando en el campo precio el valor 50, luego el método find filtra todos los libros cuyo precio sean exactamente igual a 50.

Otra forma de expresar la búsqueda de todos los libros con un precio igual a 50 es:

```
db.libros.find({ precio: { $eq : 50 } })
```

Es decir que luego del campo precio pasamos otro objeto literal iniciando el operador $eq con el valor 50.

Mostramos esta segunda forma de consultar todos los libros con un precio igual a 50 debido a que cuando tenemos que consultar por ejemplo los libros con un precio inferior a 50, o superior a 50 etc. debemos indicar en forma obligatoria el operador a utilizar.

Para mostrar todos los libros con un precio inferior a 30 tenemos que utilizar el operador $lt:

```
db.libros.find({ precio: { $lt : 30 } })
```

Tenemos luego como resultado con los datos ya cargados en la colección libros:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
{ "acknowledged" : true, "insertedId" : 3 }
> db.libros.insertOne(
...     {
...         _id: 4,
...         titulo: 'Java en 10 minutos',
...         editorial: ['Siglo XXI'],
...         precio: 45,
...         cantidad: 1
...     }
... )
{ "acknowledged" : true, "insertedId" : 4 }
>
> db.libros.find({ precio: { $lt : 30 } })
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" : [ "
Siglo XXI", "Planeta" ], "precio" : 20, "cantidad" : 50 }
>
```

Hay un solo libro que tiene un precio menor a 30.

Listado de operadores relacionales

- $eq - equal - igual
- $lt - low than - menor que
- $lte - low than equal - menor o igual que
- $gt - greater than - mayor que
- $gte - greater than equal - mayor o igual que
- $ne - not equal - distinto
- $in - in - dentro de
- $nin - not in - no dentro de

Veamos con algunos ejemplos como utilizar estos operadores para recuperar documentos que cumplen determinadas condiciones.

Recuperar todos los libros que tienen un precio mayor a 40:

```
db.libros.find({ precio: { $gt:40 }})
```

Recuperar todos los libros que en le campo cantidad tiene 50 o más:

```
db.libros.find( { cantidad: { $gte : 50 }})
```

Recuperar todos los libros que en le campo cantidad hay un valor distinto a 50:

```
db.libros.find( { cantidad: { $ne : 50 }})
```

Recuperar todos los libros cuyo precio estén comprendidos entre 20 y 45:

```
db.libros.find( { precio: { $gte : 20 , $lte : 45} })
```

Recuperar todos los libros de la editorial 'Planeta':

```
db.libros.find( { editorial: { $in : ['Planeta'] } })
```

Recuperar todos los libros que no pertenezcan a la editorial 'Planeta':

```
db.libros.find( { editorial: { $nin : ['Planeta'] } })
```

Hay que acostumbrarse en un principio a utilizar estos operados para filtrar documentos de una colección, luego veremos que estos operadores también se emplean cuando efectuemos borrados y modificaciones de documentos.

10 - Borrar documentos de una colección con los métodos deleteOne y deleteMany

Hemos visto en conceptos anteriores que podemos eliminar todos los documentos de una colección mediante el método deleteMany y pasando un objeto literal vacío:

```
db.libros.deleteMany({})
```

Aprendimos también a recuperar algunos documentos con el método find empleando una serie de operadores relacionales, dichos operadores se pueden emplear en forma idéntica con los métodos deleteMany y deleteOne.

Hay dos métodos para eliminar documentos:

- deleteMany: Borra todos los documentos que cumplen la condición que le enviamos.
- deleteOne: Borra el primer documento que cumple la condición que le pasamos.

Almacenaremos una serie de documentos en una colección llamada libros y luego borraremos algunos de sus documentos:

```
use base1
db.libros.drop()

db.libros.insertOne(
  {
    _id: 1,
    titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
    cantidad: 50
  }
)
db.libros.insertOne(
  {
    _id: 2,
    titulo: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
```

```
    precio: 50,
    cantidad: 12
  }
)
db.libros.insertOne(
  {
    _id: 3,
    titulo: 'Aprenda PHP',
    autor: 'Mario Molina',
    editorial: ['Siglo XXI','Planeta'],
    precio: 50,
    cantidad: 20
  }
)
db.libros.insertOne(
  {
    _id: 4,
    titulo: 'Java en 10 minutos',
    editorial: ['Siglo XXI'],
    precio: 45,
    cantidad: 1
  }
)

db.libros.find()

db.libros.deleteOne({_id: 1})

db.libros.find()

db.libros.deleteMany({precio : {$gte : 50 }})

db.libros.find()
```

Si queremos eliminar el documento que almacena en el campo el _id con valor 1 luego podemos utilizar la sintaxis:

```
db.libros.deleteOne({_id: 1})
```

Lo más conveniente es utilizar el método 'deleteOne' ya que solo uno puede cumplir esa condición al ser la clave primaria del documento.

Recordemos que la sintaxis alternativa para eliminar el documento con _id con valor 1 es:

```
db.libros.deleteOne({_id: { $eq : 1}})
```

La sintaxis anterior es buena recordar ya que los otros operadores relacionales hay que utilizarlos en forma obligatoria y no tienen una sintaxis alternativa como el $eq.

Para borrar todos los libros que tienen un precio mayor o igual a 50 tenemos:

```
db.libros.deleteMany({precio : {$gte : 50 }})
```

La ejecución del método deleteOne y deleteMany informa la cantidad de documentos eliminados:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
>
> db.libros.deleteOne({_id: 1})
{ "acknowledged" : true, "deletedCount" : 1 }
>
> db.libros.find()
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "editorial"
 : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "editorial" : [
 "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo XXI" ], "pr
ecio" : 45, "cantidad" : 1 }
>
> db.libros.deleteMany({precio : {$gte : 50 }})
{ "acknowledged" : true, "deletedCount" : 2 }
>
> db.libros.find()
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo XXI" ], "pr
ecio" : 45, "cantidad" : 1 }
>
```

11 - Modificar un documento mediante el método updateOne

Hemos visto en conceptos anteriores como insertar un documento en una colección, recuperar un documento, borrar un documento y nos está faltando otra operación fundamental que podemos hacer con un documento que es su modificación.

Para modificar un documento en particular disponemos de un método llamado updateOne, veamos con un ejemplo algunas de sus posibilidades:

```
use base1
db.libros.drop()

db.libros.insertOne(
  {
    _id: 1,
    titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
    cantidad: 50
  }
)
db.libros.insertOne(
  {
    _id: 2,
    titulo: 'Martin Fierro',
```

```
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
    precio: 50,
    cantidad: 12
  }
)
db.libros.insertOne(
  {
    _id: 3,
    titulo: 'Aprenda PHP',
    autor: 'Mario Molina',
    editorial: ['Siglo XXI','Planeta'],
    precio: 50,
    cantidad: 20
  }
)
db.libros.insertOne(
  {
    _id: 4,
    titulo: 'Java en 10 minutos',
    editorial: ['Siglo XXI'],
    precio: 45,
    cantidad: 1
  }
)

db.libros.find()

db.libros.updateOne({_id : {$eq:1}} , {$set : {precio:15,cantidad:1} })

db.libros.find()
```

Hemos llamado al método updateOne con dos parámetros, el primero indica el documento a modificar y el segundo parámetro utilizamos el operador $set que es un operador de actualización seguidamente con los campos y valores a modificar.

Si existe el documento con _id igual a 1 luego se modifican los campos precio y cantidad:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
>
> db.libros.updateOne({_id : {$eq:1}} , {$set : {precio:15,cantidad:1}
})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
>
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" :
[ "Siglo XXI", "Planeta" ], "precio" : 15, "cantidad" : 1 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "
editorial" : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "edit
orial" : [ "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo X
XI" ], "precio" : 45, "cantidad" : 1 }
>
```

Con las bases de datos documentales tengamos en cuenta que los documentos pueden tener distintas cantidades de campos. Por ejemplo si queremos agregar el campo descripción al libro con '_id' 4 debemos utilizar la sintaxis:

```
db.libros.updateOne({_id: {$eq:4}} ,{$set : {descripcion: 'Cada unidad trata un
tema fundamental de Java desde 0.'} })
```

Luego de ejecutar tenemos el documento con _id 4 que contiene un nuevo campo llamado 'descripcion' con el valor 'Cada unidad trata un tema fundamental de Java desde 0.':

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
XI" ], "precio" : 45, "cantidad" : 1 }
> db.libros.updateOne({_id: {$eq:4}} ,{$set : {descripcion: 'Cada unid
ad trata un tema fundamental de Java desde 0.'} })
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" :
[ "Siglo XXI", "Planeta" ], "precio" : 15, "cantidad" : 1 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "
editorial" : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "edit
orial" : [ "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo X
XI" ], "precio" : 45, "cantidad" : 1, "descripcion" : "Cada unidad tra
ta un tema fundamental de Java desde 0." }
>
```

Si queremos eliminar un campo de un documento debemos emplear el operador de actualización $unset. Probemos ahora de eliminar el campo que acabamos de crear para el documento con _id 4:

```
db.libros.updateOne({_id : {$eq:4}} , {$unset : {descripcion:''} })
```

Luego de ejecutar el updateOne tenemos que para el documento que tiene el _id 4 se ha eliminado el campo 'descripcion':

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.libros.updateOne({_id : {$eq:4}} , {$unset : {descripcion:''} })
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" :
[ "Siglo XXI", "Planeta" ], "precio" : 15, "cantidad" : 1 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "
editorial" : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "edit
orial" : [ "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo X
XI" ], "precio" : 45, "cantidad" : 1 }
>
```

Es importante entender que mediante el operador $unset eliminamos el campo, en cambio si utilizamos el operador $set modificamos el contenido del campo, luego si ejecutamos:

```
db.libros.updateOne({_id : {$eq:4}} , {$set : {descripcion:''} })
```

El campo 'descripcion' sigue existiendo y almacena una cadena de texto vacía, y si no existía se crea con una cadena vacía.

Disponemos también de operadores de modificación para arreglos, veamos como podemos agregar y eliminar elementos en el arreglo 'editorial':

```
use base1
db.libros.drop()

db.libros.insertOne(
  {
    _id: 1,
    titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
    cantidad: 50
  }
)
db.libros.insertOne(
  {
    _id: 2,
    titulo: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
    precio: 50,
    cantidad: 12
  }
)
db.libros.insertOne(
  {
    _id: 3,
    titulo: 'Aprenda PHP',
    autor: 'Mario Molina',
    editorial: ['Siglo XXI','Planeta'],
    precio: 50,
    cantidad: 20
  }
)
db.libros.insertOne(
  {
    _id: 4,
    titulo: 'Java en 10 minutos',
    editorial: ['Siglo XXI'],
    precio: 45,
    cantidad: 1
  }
)

db.libros.find()

db.libros.updateOne({_id : {$eq:1}} , {$push : {editorial:'Atlántida'} })
```

```
db.libros.find()
```

Podemos ver que luego de ejecutarse el método updateOne el arreglo 'editorial' tiene una nueva componente para el documento con _id 1:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.libros.updateOne({_id : {$eq:1}} , {$push : {editorial:'Atlántida
'} })
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
>
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" :
[ "Siglo XXI", "Planeta", "Atlántida" ], "precio" : 20, "cantidad" : 5
0 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "
editorial" : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "edit
orial" : [ "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo X
XI" ], "precio" : 45, "cantidad" : 1 }
>
```

De forma similar para eliminar un elemento del arreglo debemos emplear el operador $pull:

```
db.libros.updateOne({_id : {$eq:1}} , {$pull : {editorial:'Atlántida'} })
```

Podemos consultar todos los operadores de modificación en la página oficial de MongoDB.

12 - Modificar múltiples documentos con el método updateMany

Vimos en el concepto anterior que MongoDB nos provee de un método que nos permite modificar un único documento llamado updateOne.

El segundo método que nos permite actualizar documentos pero en forma masiva es el método updateMany.

Veamos con un ejemplo algunas variantes del método updateMany:

```
use base1
db.libros.drop()

db.libros.insertOne(
  {
    _id: 1,
    titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
```

```
    cantidad: 50
  }
)
db.libros.insertOne(
  {
    _id: 2,
    titulo: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
    precio: 50,
    cantidad: 12
  }
)
db.libros.insertOne(
  {
    _id: 3,
    titulo: 'Aprenda PHP',
    autor: 'Mario Molina',
    editorial: ['Siglo XXI','Planeta'],
    precio: 50,
    cantidad: 20
  }
)
db.libros.insertOne(
  {
    _id: 4,
    titulo: 'Java en 10 minutos',
    editorial: ['Siglo XXI'],
    precio: 45,
    cantidad: 1
  }
)

db.libros.find()

db.libros.updateMany({_id : {$gt:2}} , {$set : {cantidad:0} })

db.libros.find()

db.libros.updateMany({cantidad : {$eq:0}} , {$set : {faltantes:true} })

db.libros.find()

db.libros.updateMany({cantidad : {$eq:0}} , {$unset : {faltantes:true}, $set:
{cantidad: 100}} )

db.libros.find()
```

La primer modificación masiva la hacemos con todos los libros cuyo _id sean mayores a 2, fijando el campo cantidad con 0:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
>
> db.libros.updateMany({_id : {$gt:2}} , {$set : {cantidad:0} })
{ "acknowledged" : true, "matchedCount" : 2, "modifiedCount" : 2 }
>
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" : [
 "Siglo XXI", "Planeta" ], "precio" : 20, "cantidad" : 50 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "e
ditorial" : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "edito
rial" : [ "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 0 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo XX
I" ], "precio" : 45, "cantidad" : 0 }
>
```

La segunda modificación masiva la hacemos con todos los libros que almacenan en el campo 'cantidad' el valor cero, agregando el campo 'faltantes' con el valor true:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
>
> db.libros.updateMany({cantidad : {$eq:0}} , {$set : {faltantes:true} })
{ "acknowledged" : true, "matchedCount" : 2, "modifiedCount" : 2 }
>
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" : [ "S
iglo XXI", "Planeta" ], "precio" : 20, "cantidad" : 50 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "edit
orial" : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "editoria
l" : [ "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 0, "faltantes
" : true }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo XXI"
], "precio" : 45, "cantidad" : 0, "faltantes" : true }
>
```

La tercer y última modificación masiva la hacemos con todos los libros que almacenan en el campo 'cantidad' el valor cero, eliminamos el campo faltantes y fijamos el campo cantidad con el valor 100:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
>
> db.libros.updateMany({cantidad : {$eq:0}} , {$unset : {faltantes:true}
, $set:{cantidad: 100}} )
{ "acknowledged" : true, "matchedCount" : 2, "modifiedCount" : 2 }
>
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" : [
"Siglo XXI", "Planeta" ], "precio" : 20, "cantidad" : 50 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "ed
itorial" : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "editor
ial" : [ "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 100 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo XXI
" ], "precio" : 45, "cantidad" : 100 }
>
```

13 - Operadores lógicos $and, $or y $not

Cuando necesitamos construir consultas que deban cumplir varias condiciones utilizaremos los operadores lógicos.

El operador $and lo hemos utilizando en forma implícita, por ejemplo si tenemos:

```
db.libros.find({precio : 50, cantidad : 20 })
```

Con la condición anterior se recuperan todos los libros que tienen un precio de 50 y la cantidad es 20. Las dos condiciones deben ser verdaderas para que el documento se recupere.

La sintaxis alternativa para el find es:

```
db.libros.find({$and : [{precio:50}, {cantidad:20}]  })
```

El valor para el operador $and es un arreglo con cada una de las condiciones que debe cumplir.

Para los operadores $or y $not no hay una forma de disponer una sintaxis implícita.

Veamos con ejemplos el empleo de los operadores $or y $not:

```
use base1
db.libros.drop()

db.libros.insertOne(
  {
    _id: 1,
    titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
    cantidad: 50
  }
```

```
)
db.libros.insertOne(
  {
    _id: 2,
    titulo: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
    precio: 50,
    cantidad: 12
  }
)
db.libros.insertOne(
  {
    _id: 3,
    titulo: 'Aprenda PHP',
    autor: 'Mario Molina',
    editorial: ['Siglo XXI','Planeta'],
    precio: 50,
    cantidad: 20
  }
)
db.libros.insertOne(
  {
    _id: 4,
    titulo: 'Java en 10 minutos',
    editorial: ['Siglo XXI'],
    precio: 45,
    cantidad: 1
  }
)

db.libros.find()

db.libros.find({$or: [{precio:{$gte:50}}, {cantidad:1} ]})
```

Para recuperar los libros que tienen un precio mayor o igual a 50 o la cantidad es 1 debemos implementar mediante un $or la siguiente sintaxis:

```
db.libros.find({$or: [{precio:{$gte:50}}, {cantidad:1} ]})
```

Hay tres documentos que cumplen la condición:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
>
> db.libros.find()
{ "_id" : 1, "titulo" : "El aleph", "autor" : "Borges", "editorial" : [ "Siglo X
XI", "Planeta" ], "precio" : 20, "cantidad" : 50 }
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "editorial"
 : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "editorial" : [
 "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo XXI" ], "pr
ecio" : 45, "cantidad" : 1 }
>
> db.libros.find({$or: [{precio:{$gte:50}}, {cantidad:1} ]})
{ "_id" : 2, "titulo" : "Martin Fierro", "autor" : "Jose Hernandez", "editorial"
 : [ "Siglo XXI" ], "precio" : 50, "cantidad" : 12 }
{ "_id" : 3, "titulo" : "Aprenda PHP", "autor" : "Mario Molina", "editorial" : [
 "Siglo XXI", "Planeta" ], "precio" : 50, "cantidad" : 20 }
{ "_id" : 4, "titulo" : "Java en 10 minutos", "editorial" : [ "Siglo XXI" ], "pr
ecio" : 45, "cantidad" : 1 }
>
```

Si queremos recuperar todos los documentos de la colección libros que no tienen un precio mayor o igual a 50 la sintaxis debe ser:

```
db.libros.find({precio: {$not:{$gte:50}} })
```

Los operadores lógicos podemos utilizarlos no solo para recuperar datos, sino también cuando borramos o actualizamos documentos.

Si queremos borrar todos los libros cuyo precio no sean iguales a 50 podemos codificar:

```
db.libros.deleteMany({precio: {$not:{$eq:50}} })
```

Se eliminan dos documentos de la colección libros.

15 - Recuperar algunos campos de una colección con el método find

Hemos visto que el método 'find':

- Si no le pasamos parámetros nos retorna todos los documentos de la colección que hace referencia:

  ```
  db.libros.find()
  ```

- El primer parámetro en el caso que lo indiquemos filtra la colección y recupera los documentos que cumplen la condición:

```
db.libros.find({precio : 50 })
```

Recuperamos todos los libros que tienen un precio igual a 50.

- Hay un segundo parámetro opcional en el cual debemos indicar que campos del documento queremos recuperar:

```
use base1
db.libros.drop()

db.libros.insertOne(
  {
    _id: 1,
    titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
    cantidad: 50
  }
)
db.libros.insertOne(
  {
    _id: 2,
    titulo: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
    precio: 50,
    cantidad: 12
  }
)
db.libros.insertOne(
  {
    _id: 3,
    titulo: 'Aprenda PHP',
    autor: 'Mario Molina',
    editorial: ['Siglo XXI','Planeta'],
    precio: 50,
    cantidad: 20
  }
)
db.libros.insertOne(
  {
    _id: 4,
    titulo: 'Java en 10 minutos',
    editorial: ['Siglo XXI'],
    precio: 45,
    cantidad: 1
  }
)

db.libros.find({precio : 50 },{titulo:1,cantidad:1})
```

En el segundo parámetro del método 'find' debemos especificar cada campo y un valor 1 indicando que se lo quiere recuperar. El campo _id se recupera por defecto, salvo que indiquemos con un valor 0 para que no se recupere:

Solo se recuperan los campos titulo,cantidad y _id.

16 - Documentos embebidos: definición de campos de tipo documento

Hasta ahora hemos trabajado definiendo documentos con una serie de campos que almacenan tipo de datos simples como enteros, reales, cadenas de caracteres y datos compuestos de tipo arreglo.

Según el tipo de consultas que haremos a nuestros documentos debemos definir el esquema de campos de los mismos. Supongamos que tenemos que almacenar el dato de clientes y luego hacer consultas de donde viven y discriminar por calle y número, luego puede tener sentido tener un campo dirección que sea un documento donde se almacenen por separado la calle, el número y el código postal.

Veamos con un ejemplo como creamos una colección de documentos utilizando documentos embebidos.

```
use base1
db.clientes.drop()

db.clientes.insertOne(
  {
    _id: 1,
    nombre: 'Martinez Victor',
    mail: 'mvictor@gmail.com',
    direccion: {
      calle: 'Colon',
      numero: 620,
      codigopostal: 5000
    }
  }
)
db.clientes.insertOne(
  {
    _id: 2,
    nombre: 'Alonso Carlos',
    mail: 'acarlos@gmail.com',
    direccion: {
      calle: 'Colon',
      numero: 150,
      codigopostal: 5000
    }
  }
```

```
)
db.clientes.insertOne(
  {
    _id: 3,
    nombre: 'Gonzalez Marta',
    mail: 'gmarta@outlook.com',
    direccion: {
      calle: 'Colon',
      numero: 1200,
      codigopostal: 5000
    }
  }
)
db.clientes.insertOne(
  {
    _id: 4,
    nombre: 'Ferrero Ariel',
    mail: 'fariel@yahoo.com',
    direccion: {
      calle: 'Dean Funes',
      numero: 23,
      codigopostal: 5002
    }
  }
)
db.clientes.insertOne(
  {
    _id: 5,
    nombre: 'Fernandez Diego',
    mail: 'fdiego@gmail.com',
    direccion: {
      calle: 'Dean Funes',
      numero: 561,
      codigopostal: 5002
    }
  }
)

db.clientes.find()
```

Para recuperar todos los clientes que viven en la calle 'Colon' tenemos que plantear la siguiente consulta:

```
db.clientes.find({'direccion.calle':'Colon'}).pretty()
```

Es obligatorio disponer las comillas cuando hacemos referencia a un subcampo de un documento: 'direccion.calle'.

El resultado de ejecutar la consulta es:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.clientes.find({'direccion.calle':'Colon'}).pretty()
{
        "_id" : 1,
        "nombre" : "Martinez Victor",
        "mail" : "mvictor@gmail.com",
        "direccion" : {
                "calle" : "Colon",
                "numero" : 620,
                "codigopostal" : 5000
        }
}
{
        "_id" : 2,
        "nombre" : "Alonso Carlos",
        "mail" : "acarlos@gmail.com",
        "direccion" : {
                "calle" : "Colon",
                "numero" : 150,
                "codigopostal" : 5000
        }
}
{
        "_id" : 3,
        "nombre" : "Gonzalez Marta",
        "mail" : "gmarta@outlook.com",
        "direccion" : {
                "calle" : "Colon",
                "numero" : 1200,
                "codigopostal" : 5000
        }
}
>
```

Para recuperar todos los clientes que viven en la calle 'Colon' y su número está comprendido entre 1 y 1000 tenemos que plantear la siguiente consulta:

```
db.clientes.find({ 'direccion.calle':'Colon','direccion.numero':
{$gte:0},'direccion.numero':{$lte:1000} }).pretty()
```

Si ejecutamos la consulta podemos comprobar que hay dos documentos que cumplen la condición impuesta al método 'find':

```
C:\Program Files\MongoDB\Server\4.0\b...
'direccion.numero':{$gte:0},'direccion.numero':
{$lte:1000} }).pretty()
{
        "_id" : 1,
        "nombre" : "Martinez Victor",
        "mail" : "mvictor@gmail.com",
        "direccion" : {
                "calle" : "Colon",
                "numero" : 620,
                "codigopostal" : 5000
        }
}
{
        "_id" : 2,
        "nombre" : "Alonso Carlos",
        "mail" : "acarlos@gmail.com",
        "direccion" : {
                "calle" : "Colon",
                "numero" : 150,
                "codigopostal" : 5000
        }
}
>
```

17 - Documentos embebidos: definición de campos de arreglo con elementos de tipo documento

Una de las ventajas fundamentales del gestor de base de datos MongoDB es la posibilidad de crear esquemas de documentos muy flexibles y no tener que guardar datos relacionados en distintos documentos.

Desde los primeros ejemplos hemos visto que podemos definir campos en un documento de tipo arreglo:

```
db.libros.insertOne(
  {
    codigo: 1,
    nombre: 'El aleph',
    autor: 'Borges',
    editoriales: ['Planeta','Siglo XXI']
  }
)
```

El campo 'editoriales' es de tipo arreglo y almacena dos cadenas de caracteres.

Podemos definir campos de tipo arreglo y almacenar en los mismos documentos embebidos, por ejemplo:

```
use base1
db.posts.drop()

db.posts.insertOne(
  {
    _id: 1,
    titulo: 'Lenguaje Java',
    contenido: 'Uno de los lenguajes más utilizados es ...',
    comentarios: [{
        autor: 'Marcos Paz',
        mail: 'pazm@gmail.com',
        contenido: 'Me parece un buen...'
    },
    {
        autor: 'Ana Martinez',
        mail: 'martineza@gmail.com',
        contenido: 'Todo ha cambiado en...'
    },
    {
        autor: 'Luiz Blanco',
        mail: 'blancol@outlook.com',
        contenido: 'Afirmo que es...'
    }]
  }
)

db.posts.find().pretty()
```

El campo comentarios almacena un arreglo de tres elementos, cada elemento es un documento.

La ejecución de este bloque:

C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe

```
> use base1
switched to db base1
> db.posts.drop()
true
>
> db.posts.insertOne(
...   {
...     _id: 1,
...     titulo: 'Lenguaje Java',
...     contenido: 'Uno de los lenguajes más utilizados es ...',
...     comentarios: [{
...         autor: 'Marcos Paz',
...         mail: 'pazm@gmail.com',
...         contenido: 'Me parece un buen...'
...     },
...     {
...         autor: 'Ana Martinez',
...         mail: 'martineza@gmail.com',
...         contenido: 'Todo ha cambiado en...'
...     },
...     {
...         autor: 'Luiz Blanco',
...         mail: 'blancol@outlook.com',
...         contenido: 'Afirmo que es...'
...     }]
...   }
... )
{ "acknowledged" : true, "insertedId" : 1 }
>
> db.posts.find().pretty()
{
        "_id" : 1,
        "titulo" : "Lenguaje Java",
        "contenido" : "Uno de los lenguajes más utilizados es ...",
        "comentarios" : [
                {
                        "autor" : "Marcos Paz",
                        "mail" : "pazm@gmail.com",
                        "contenido" : "Me parece un buen..."
                },
                {
                        "autor" : "Ana Martinez",
                        "mail" : "martineza@gmail.com",
                        "contenido" : "Todo ha cambiado en..."
                },
                {
                        "autor" : "Luiz Blanco",
                        "mail" : "blancol@outlook.com",
```

La idea fundamental en MongoDB es disponer de todos los datos agrupados dependiendo de los accesos futuros a la información, es evidente que cuando queremos acceder a una entrada de un bloq también necesitemos acceder a los comentarios de dicha entrada.

Esta forma de organizar los datos nos permite implementar aplicaciones muy eficientes en comparación al modelo de base de datos relacional (MySQL, Oracle, SQL Server etc.)

Podemos generar documentos embebidos con más niveles, por ejemplo definir un campo en el arreglo que sea de tipo documento.

Consultas de los documentos embebidos

Ejecutemos el siguiente bloque de comandos en MongoDB para analizar las consultas en documentos embebidos dentro de arreglos:

```
use base1
db.posts.drop()

db.posts.insertOne(
  {
    _id: 1,
    titulo: 'Lenguaje Java',
    contenido: 'Uno de los lenguajes más utilizados es ...',
    comentarios: [{
        autor: 'Marcos Paz',
        mail: 'pazm@gmail.com',
        contenido: 'Me parece un buen...'
    },
    {
        autor: 'Ana Martinez',
        mail: 'martineza@gmail.com',
        contenido: 'Todo ha cambiado en...'
    },
    {
        autor: 'Luiz Blanco',
        mail: 'blancol@outlook.com',
        contenido: 'Afirmo que es...'
    }]
  }
)

db.posts.insertOne(
  {
    _id: 2,
    titulo: 'Lenguaje C#',
    contenido: 'Microsoft desarrolla el lenguaje C# con el objetivo ...',
    comentarios: [{
        autor: 'Pablo Rodriguez',
        mail: 'rodriguezp@gmail.com',
        contenido: 'Correcta idea.'
    },
    {
        autor: 'Maria Contreras',
        mail: 'contrerasm@gmail.com',
        contenido: 'Buen punto de vista...'
    }]
```

```
  }
)
```

```
db.posts.find({},{_id:0,titulo:1,'comentarios.autor':1}).pretty()
```

```
db.posts.find({'comentarios.autor':'Pablo Rodriguez'}).pretty()
```

```
db.posts.find({'comentarios.0.autor':'Pablo Rodriguez'}).pretty()
```

Si queremos recuperar los títulos de todos los posts y los nombres de los usuarios que hicieron comentarios en cada post, debemos implementar la siguiente consulta:

```
db.posts.find({},{_id:0,titulo:1,'comentarios.autor':1}).pretty()
```

Y como resultado tenemos:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.posts.find({},{_id:0,titulo:1,'comentarios.autor':1}).pretty()
{
        "titulo" : "Lenguaje Java",
        "comentarios" : [
                {
                        "autor" : "Marcos Paz"
                },
                {
                        "autor" : "Ana Martinez"
                },
                {
                        "autor" : "Luiz Blanco"
                }
        ]
}
{
        "titulo" : "Lenguaje C#",
        "comentarios" : [
                {
                        "autor" : "Pablo Rodriguez"
                },
                {
                        "autor" : "Maria Contreras"
                }
        ]
}
>
```

Para recuperar todos los posts donde ha comentado el usuario 'Pablo Rodriguez':

```
db.posts.find({'comentarios.autor':'Pablo Rodriguez'}).pretty()
```

Y como resultado tenemos:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.posts.find({'comentarios.autor':'Pablo Rodriguez'}).pretty()
{
        "_id" : 2,
        "titulo" : "Lenguaje C#",
        "contenido" : "Microsoft desarrolla el lenguaje C# con el ob
jetivo ...",
        "comentarios" : [
                {
                        "autor" : "Pablo Rodriguez",
                        "mail" : "rodriguezp@gmail.com",
                        "contenido" : "Correcta idea."
                },
                {
                        "autor" : "Maria Contreras",
                        "mail" : "contrerasm@gmail.com",
                        "contenido" : "Buen punto de vista..."
                }
        ]
}
>
```

Podemos utilizar el subíndice del arreglo para analizar un documento en particular, por ejemplo si queremos recuperar todos los posts que comentó primero 'Rodriguez Pablo':

```
db.posts.find({'comentarios.0.autor':'Pablo Rodriguez'}).pretty()
```

Acotaciones

Cuando definimos documentos embebidos debemos tener en cuenta que un documento en MongoDB no puede tener un tamaño mayor a 16MB.

No podemos crear una colección 'ciudades' y almacenar documentos con el nombre de la ciudad y embeber documentos con los nombres de todos sus ciudadanos (podría haber millones por cada ciudad)

En el ejemplo de la colección 'posts' los comentarios por cada post serán una cantidad limitada y no millones.

Cuando uno define el esquema de los documentos a almacenar en MongoDB debe tener en cuenta cuanto pueden crecer los documentos embebidos para tomar la decisión de separar los mismos en documentos independientes.

18 - Campo _id generado por MongoDB

Cuando creamos un documento podemos o no iniciar el campo _id, en la mayoría de los ejemplos hemos definido nosotros el campo _id y su valor, recordemos que no pueden repetirse.

Si no definimos nosotros el campo _id se genera en forma automática:

```
use base1
db.libros.drop()

db.libros.insertOne(
  {
     titulo: 'El aleph',
    autor: 'Borges',
    editorial: ['Siglo XXI','Planeta'],
    precio: 20,
    cantidad: 50
  }
)
db.libros.insertOne(
  {
    titulo: 'Martin Fierro',
    autor: 'Jose Hernandez',
    editorial: ['Siglo XXI'],
    precio: 50,
    cantidad: 12
  }
)

db.libros.find()
```

Tenemos como resultado:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> db.libros.find().pretty()
{
        "_id" : ObjectId("5c39e2c7687b2322fe53eba6"),
        "titulo" : "El aleph",
        "autor" : "Borges",
        "editorial" : [
                "Siglo XXI",
                "Planeta"
        ],
        "precio" : 20,
        "cantidad" : 50
}
{
        "_id" : ObjectId("5c39e2c7687b2322fe53eba7"),
        "titulo" : "Martin Fierro",
        "autor" : "Jose Hernandez",
        "editorial" : [
                "Siglo XXI"
        ],
        "precio" : 50,
        "cantidad" : 12
}
>
```

El valor que se genera es de la clase ObjectId y tiene la característica de poder generarse con un valor único aunque se esté generando en servidores distintos.

Uno de los objetivos fundamentales de emplear MongoDB es poder tener servidores distribuidos, la posibilidad de definir una clave primaria única y no tener que comunicarse entre los servidores se resuelve utilizando la clase ObjectId.

No se puede generar un campo numérico autoincremental como en los gestores de bases de datos relacionales, esto obligaría a sincronizar dicho campo con otros servidores y generaría cuellos de botella y excesivo tráfico.

MongoDB fue diseñado para ser una base de datos distribuida, es fundamenta poder generar identificadores únicos en un entorno fragmentado con múltiples servidores.

Un objeto de la clase ObjectId requiere 12 bytes y representan:

- Los primeros 4 bytes indican la cantidad de segundos desde el 1 de enero de 1970 (época unix)
  Cuando se ordena por el _id luego los documentos se encontrarán organizados según el tiempo que se lo insertó.
- Los 3 bytes siguientes del ObjectId son un identificador único de la máquina en la que se generó, esto nos garantiza que los diferentes servidores no generarán ObjectId iguales.
- Los 2 bytes siguientes evitan que haya colisiones de ObjectId en el mismo servidor, el mismo se genera con el Id del número de proceso que lo generó.
- Los primeros 9 bytes de un ObjectId garantizan su unicidad en todas las máquinas y procesos durante un segundo. Los últimos 3 bytes son simplemente un contador incremental que es responsable de la unicidad dentro de un segundo en un solo proceso. Esto permite generar hasta 16.777.216 ObjectIds únicos por proceso en un solo segundo.

19 - Tipo de dato Date en MongoDB

Hemos utilizado hasta ahora varios tipos de datos cuando inicializamos campos de un documento:

- String : Permiten almacenar cadenas de caracteres en formato UTF-8
- Integer32 : Valores entero numérico
- Integer64 : Valores entero numérico
- Double : Almacena valores de punto flotante
- Object : Almacena un documento embebido
- Array : Permite almacenar un arreglo con elementos de distinto tipo
- Boolean : Permite almacenar un valor true o false

Veremos ahora como almacenar datos de tipo Date.

La fecha es un entero de 64 bits que representa el número de milisegundos desde la época de Unix (1 de enero de 1970). Esto da como resultado un intervalo de fechas representables de

aproximadamente 290 millones de años en el pasado y el futuro. El tipo Date almacena el signo, un valor negativo representa una fecha anterior a 1970.

Veamos un ejemplo como podemos almacenar un tipo de dato Date tomando la fecha y hora actual del servidor.

Problema

Una playa de estacionamiento cada vez que ingresa un vehículo crea un documento donde almacena la patente y la fecha y hora de ingreso .

```
use base1
db.autos.drop()

db.autos.insertOne(
  {
    patente : 'aaa111',
    fechahora : new Date()
  }
)
db.autos.insertOne(
  {
    patente : 'bbb222',
    fechahora : new Date()
  }
)
db.autos.insertOne(
  {
    patente : 'ccc333',
    fechahora : new Date()
  }
)

db.autos.find().pretty()
```

La representación de los datos de tipo Date cuando llamamos al método find es:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
>
> db.autos.find().pretty()
{
        "_id" : ObjectId("5c3a60d52c419f7680420488"),
        "patente" : "aaa111",
        "fechahora" : ISODate("2019-01-12T21:49:09.852Z")
}
{
        "_id" : ObjectId("5c3a60d52c419f7680420489"),
        "patente" : "bbb222",
        "fechahora" : ISODate("2019-01-12T21:49:09.873Z")
}
{
        "_id" : ObjectId("5c3a60d52c419f768042048a"),
        "patente" : "ccc333",
        "fechahora" : ISODate("2019-01-12T21:49:09.883Z")
}
>
```

Para almacenar la fecha se utiliza el estándar ISO 8601 que tiene un formato:
YYYY-MM-DDTHH:MM:SS

Podemos almacenar una fecha particular cuando creamos el objeto de la clase Date:

```
use base1
db.empleados.drop()

db.empleados.insertOne(
  {
    _id : 20456234,
    nombre : 'Rodriguez Pablo',
    fechaingreso : new Date(2010,0,31)
  }
)
db.empleados.insertOne(
  {
    _id : 17488834,
    nombre : 'Gomez Ana',
    fechaingreso : new Date(2001,11,1)
  }
)
db.empleados.insertOne(
  {
    _id : 23463564,
    nombre : 'Juarez Carla',
    fechaingreso : new Date(2005,3,14)
  }
)

db.empleados.find().pretty()
```

Como el shell de MongoDB está implementado en JavaScript debemos indicar al crear un objeto dee la clase Data para el mes un valor comprendido entre 0 y 11.

Podemos ver las fechas almacenadas en el campo 'fechaingreso', y que la parte de la hora está en cero por no pasarlas cuando creamos el objeto de la clase Date:

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
>
> db.empleados.find().pretty()
{
        "_id" : 20456234,
        "nombre" : "Rodriguez Pablo",
        "fechaingreso" : ISODate("2010-01-31T03:00:00Z")
}
{
        "_id" : 17488834,
        "nombre" : "Gomez Ana",
        "fechaingreso" : ISODate("2001-12-01T03:00:00Z")
}
{
        "_id" : 23463564,
        "nombre" : "Juarez Carla",
        "fechaingreso" : ISODate("2005-04-14T03:00:00Z")
}
>
```

Si necesitamos que los datos de empleados se recuperen en forma ordenada por el campo 'fechaingreso' debemos codificar la siguiente consulta:

```
db.empleados.find().pretty().sort({fechaingreso:1})
```

20 - Tipo de dato Binary Data

MongoDB permite definir la carga de datos binarios (por ejemplo archivos jpg, png, pdf etc.) en un campo de un documentos, siempre y cuando no supere el límite de 16Mb definido por el gestor de base de datos por motivos de eficiencia. Si tenemos que almacenar archivos muy grandes como podrían ser un archivo mp4 con una película debemos utilizar otra técnica, como por ejemplo que en el campo del documento se almacene la URL donde se encuentra la película.

Desde el shell de Mongo estamos bastante limitados para la carga de un campo de tipo binario pero podemos hacerlo creando una variable y almacenando el valor retornado por BinData:

```
use base1

db.usuarios.drop()

var datobinario=BinData(0,'e8MEnzZoFyMmD7WSHdNrFJyEk8M=')
```

```
db.usuarios.insertOne(
  {
    nombre:'marcos',
    clave:'abc123',
    fotoperfil:datobinario
  }
)

db.usuarios.find().pretty()
```

La ejecución del bloque anterior nos almacena un documento con un campo con formato 'Binary Data':

```
C:\Program Files\MongoDB\Server\4.0\bin\mongo.exe
> use base1
switched to db base1
>
> db.usuarios.drop()
true
>
> var datobinario=BinData(0,'e8MEnzZoFyMmD7WSHdNrFJyEk8M=')
>
> db.usuarios.insertOne(
...   {
...     nombre:'marcos',
...     clave:'abc123',
...     fotoperfil:datobinario
...   }
... )
{
        "acknowledged" : true,
        "insertedId" : ObjectId("5c3b3aa71d28f100acfeb465")
}
>
> db.usuarios.find().pretty()
{
        "_id" : ObjectId("5c3b3aa71d28f100acfeb465"),
        "nombre" : "marcos",
        "clave" : "abc123",
        "fotoperfil" : BinData(0,"e8MEnzZoFyMmD7WSHdNrFJyEk8M=")
}
>
```

Recordemos que en el mundo real cuando almacenemos campos de tipo Binary Data lo haremos desde una aplicación creada con alguno de los tantos lenguajes que soporta MongoDB como pueden ser: C, C++, C#, Go, Java, Node.js, Perl, PHP, Phyton, Ruby o Scala.

Las ventajas de tener los datos binarios dentro de los documentos son:

- Proporciona el motor de base de datos alta disponibilidad y replicación de datos.
- Una arquitectura de aplicación más simple al tener centralizado todos los datos.
- Cuando se usan conjuntos de réplicas distribuidas geográficamente, MongoDB distribuirá automáticamente los datos a centros de datos geográficamente distintos.
- El almacenamiento de datos en la base de datos aprovecha los mecanismos de autenticación y seguridad de MongoDB.
- Mejor rendimiento en los accesos a datos. Accediendo al documento contamos con todos los datos y no tenemos que hacer referencias a otras locaciones con datos.

Hay que tener en cuenta que un documento puede crecer solo hasta 16Mb, en el caso de ser archivos binarios con tamaños superiores hay que utilizar otra técnica para su almacenamiento, lo más común es almacenarlos la referencia mediante una URL.

Hay otra técnica de trocear el archivo y almacenarlo en una sucesión de documentos, puede consultar en la página oficial de MongoDB.